

</>

The Anuj DECODED

</>

```
01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};
```



```
01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver
```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

1621 · Number of Sets of K Non-Overlapping Line Segments

LeetCode · Daily Question

Medium

Combinatorics

DP

1. What Type of Question Is This?

This is a **counting DP** problem dressed up as geometry. It's not really about line segments in space — it's about counting how many ways you can pick k non-overlapping **intervals** out of n points on a line, where "non-overlapping" specifically *allows* touching at a shared endpoint. Whenever you're asked to **count arrangements** (not find one specific arrangement, or its length/value), and the arrangement is built by scanning left to right making a small number of local choices, a DP over (position, count-so-far, and possibly one more small state) is the standard tool.

2. What Is Given?

- n points on a 1-D line, where the i -th point (0-indexed) sits at coordinate $x = i$.
- You must choose exactly k line segments, each covering **two or more points** (so each segment has a distinct start and end, e.g. covering points 0 and 1 at minimum).
- The segments must be **non-overlapping**, but they **are allowed to share endpoints** — e.g. a segment from 0 to 1 and another from 1 to 2 are both fine together, even though they touch at point 1.
- The k segments don't need to cover every point — points can be left completely unused.

3. What Do We Need to Find?

The **number of distinct ways** to draw exactly k such non-overlapping segments, returned **modulo $10^9 + 7$** (the true count can be enormous).

Function Signature

■ C++

```
class Solution {
public:
    int numberOfSets(int n, int k) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input:  n = 4, k = 2
Output: 5
```

With 4 points (0, 1, 2, 3), the 5 ways to draw 2 non-overlapping segments are: {(0,2),(2,3)}, {(0,1),(1,3)}, {(0,1),(2,3)}, {(1,2),(2,3)}, {(0,1),(1,2)}. Notice several of these pairs share an endpoint (e.g. (0,1) and (1,2) both touch at point 1) — that's explicitly allowed.

Example 2 — a single segment

```
Input:  n = 3, k = 1
Output: 3
```

With 3 points and just 1 segment needed, the 3 ways are {(0,1)}, {(0,2)}, {(1,2)} — every possible pair of points, since a single segment trivially can't overlap with anything.

Example 3 — a large modulo case

```
Input:  n = 30, k = 7
Output: 796297179
```

The true count of ways is 3,796,297,200 — far too large to return directly. Taking that number modulo 10^9+7 gives 796297179, confirming the modulo requirement really matters even for fairly modest inputs.

Constraints

- $2 \leq n \leq 1000$
- $1 \leq k \leq n-1$

n up to 1000 means an $O(n \cdot k)$ solution (up to roughly 10^6 states) is exactly the right complexity target — comfortably fast, while anything that tries to enumerate segment combinations directly is completely out of the question.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Process points left to right, one decision at a time.** Instead of thinking about "which k pairs of points to choose" all at once, walk through the points $0, 1, 2, \dots, n-1$ in order, deciding at each point what role it plays.
- **Step 2 — Track two things: how many segments are finished, and whether you're currently "inside" one.** At any point in the scan, you either aren't part of any segment right now (**free**), or you're partway through a segment you've already started but haven't closed yet (**in segment**). Combined with how many segments you've already fully closed, that's your entire DP state.
- **Step 3 — Work out what each point can do in each state.** If you're **free**: either this point isn't used by anything (stay free), or it becomes the start of a brand-new segment (become "in segment"). If you're **in a segment**: either this point is just a pass-through point the segment continues through (stay "in segment"), or this point closes the current segment (become free, segment count $+1$), or — since endpoints can be shared — this point *both* closes the current segment *and* immediately starts a new one right here (stay "in segment", segment count $+1$).
- **Step 4 — That's the whole recurrence.** No special-casing for overlaps is needed at all — the "free / in segment" state machine makes overlapping segments structurally impossible to represent, since a new segment can only ever start once the previous one has (at least momentarily) closed.
- **Step 5 — Read off the answer.** After processing all n points, the answer is the number of ways to have exactly k segments closed *and* end in the **free** state (an unclosed, still-open segment at the very end isn't a valid completed segment).

The State Machine

Two states per point: "free" or "inside an open segment"

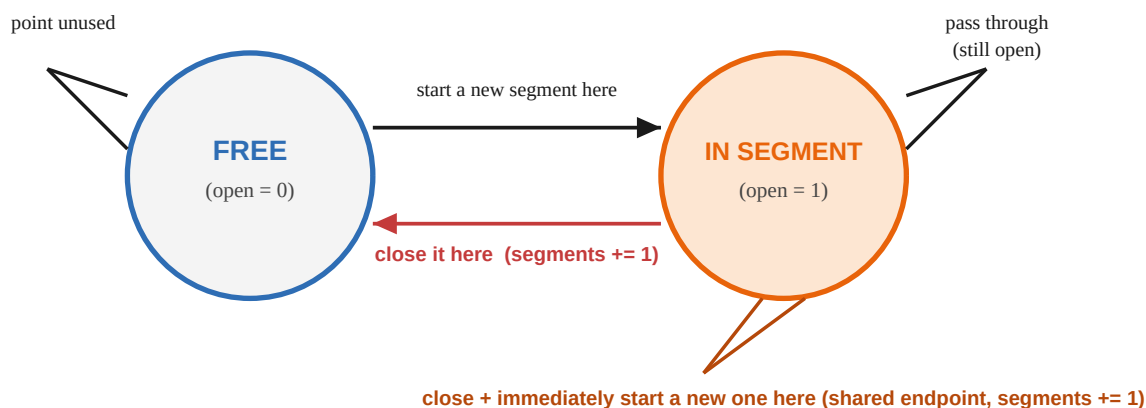


Figure 1 — every point transitions between exactly these two states; overlapping segments are structurally impossible to represent, so no separate overlap check is ever needed.

Which Algorithm Is Best?

The DP state is (point index, segments closed so far, free/in-segment) — $O(n \cdot k \cdot 2)$ states, each computed in $O(1)$ from the previous point's states. That's $O(n \cdot k)$ time overall, easily fast enough for n, k up to 1000. The only real refinement left is memory: since each point's values depend only on the *immediately preceding* point, the full

$O(n \cdot k)$ table can be collapsed down to $O(k)$ space with a rolling pair of arrays. The brute-force and "better" tiers below show the progression from direct enumeration, to the full un-rolled DP table, to that final memory-optimized version.

Approach 1 — Brute Force (Enumerate Every Combination of k Segments)

Idea: The most literal reading of the problem: list every possible segment (every pair of points), try every combination of k of them, and count how many combinations turn out to be pairwise non-overlapping once sorted by start position.

Steps

- Build the list of all $O(n^2)$ possible segments (a, b) with $a < b$.
- Generate every combination of k segments from that list.
- For each combination, sort the chosen segments by start position and check that each one ends at or before the next one starts.
- Count the combinations that pass the check, taking the result modulo 10^9+7 .

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;

class Solution {
public:
    int numberOfSets(int n, int k) {
        // Generate every possible segment (a, b) with a < b, then try every combination
        // of k of them, checking pairwise non-overlap directly.
        vector<pair<int,int>> segs;
        for (int a = 0; a < n; a++)
            for (int b = a + 1; b < n; b++)
                segs.push_back({a, b});

        int m = segs.size();
        vector<int> combo(k);
        long long count = 0;

        function<void(int,int)> choose = [&](int start, int filled) {
            if (filled == k) {
                vector<pair<int,int>> chosen;
                for (int idx : combo) chosen.push_back(segs[idx]);
                sort(chosen.begin(), chosen.end());
                bool ok = true;
                for (int i = 0; i + 1 < k; i++)
                    if (chosen[i].second > chosen[i+1].first) { ok = false; break; }
                if (ok) count = (count + 1) % MOD;
                return;
            }
            for (int i = start; i < m; i++) {
                combo[filled] = i;
                choose(i + 1, filled + 1);
            }
        };
        choose(0, 0);
        return (int)count;
    }
};

int main() {
    int n = 4, k = 2;
    Solution sol;
    cout << sol.numberOfSets(n, k) << endl;    // Expected output: 5
    return 0;
}
```

Python

```
from itertools import combinations

MOD = 10**9 + 7

class Solution:
    def numberOfSets(self, n, k):
        # Generate every possible segment (a, b) with a < b, then try every combination
        # of k of them, checking pairwise non-overlap directly.
        segs = [(a, b) for a in range(n) for b in range(a + 1, n)]

        count = 0
        for combo in combinations(segs, k):
            chosen = sorted(combo)
            ok = True
            for i in range(k - 1):
                if chosen[i][1] > chosen[i + 1][0]:
                    ok = False
                    break
            if ok:
                count = (count + 1) % MOD

        return count

if __name__ == "__main__":
    n, k = 4, 2
    print(Solution().numberOfSets(n, k))    # Expected output: 5
```

JavaScript

```
const MOD = 1000000007n;

class Solution {
    numberOfSets(n, k) {
        // Generate every possible segment (a, b) with a < b, then try every combination
        // of k of them, checking pairwise non-overlap directly.
        const segs = [];
        for (let a = 0; a < n; a++)
            for (let b = a + 1; b < n; b++) segs.push([a, b]);

        let count = 0n;
        const combo = [];

        const choose = (start) => {
            if (combo.length === k) {
                const chosen = combo.map(i => segs[i]).sort((x, y) => x[0] - y[0]);
                let ok = true;
                for (let i = 0; i + 1 < k; i++) {
                    if (chosen[i][1] > chosen[i + 1][0]) { ok = false; break; }
                }
                if (ok) count = (count + 1n) % MOD;
                return;
            }
            for (let i = start; i < segs.length; i++) {
                combo.push(i);
                choose(i + 1);
                combo.pop();
            }
        };

        choose(0);
        return Number(count);
    }
}

const n = 4, k = 2;
console.log(new Solution().numberOfSets(n, k));    // Expected output: 5
```

Complexity: There are $O(n^2)$ candidate segments, and choosing k of them costs $O((n^2)^k / k!)$ in the worst sense — combinatorially explosive even for k as small as 2 or 3 once n grows past a few dozen. With n up to 1000 and k

up to 999, this is astronomically infeasible; it only illustrates the problem's literal definition on tiny inputs.

Approach 2 — Better (Full DP Table with the Free / In-Segment State)

Idea: Implement the Step 2–5 state machine directly as a DP table indexed by (point index, segments closed, free-or-open), storing every intermediate value in a full 3-D table rather than enumerating any actual segment combinations.

Steps

- Allocate $dp[i][j][open]$ for i from 0 to n , j from 0 to k , $open$ in $\{0, 1\}$.
- Base case: $dp[0][0][0] = 1$ (before any points, zero segments closed, not in a segment).
- For each point i from 1 to n and each segment count j : compute $dp[i][j][0]$ from the FREE transitions, and $dp[i][j][1]$ from the IN SEGMENT transitions, both purely in terms of $dp[i-1][*][*]$ as described in Steps 3–4.
- The answer is $dp[n][k][0]$ — k segments closed, and not left dangling open at the very end.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;

class Solution {
public:
    int numberOfSets(int n, int k) {
        // dp[i][j][open]: after processing the first i points, j segments CLOSED,
        // open = 1 if currently inside an unclosed segment.
        vector<vector<array<long long, 2>>> dp(
            n + 1, vector<array<long long, 2>>>(k + 1, {0, 0}));
        dp[0][0][0] = 1;

        for (int i = 1; i <= n; i++) {
            for (int j = 0; j <= k; j++) {
                // FREE at point i: point unused (stay free), or close+start covered below
                long long free_ = dp[i-1][j][0];
                if (j >= 1) free_ = (free_ + dp[i-1][j-1][1]) % MOD; // close, nothing new
                dp[i][j][0] = free_;

                // IN SEGMENT at point i: start new, pass through, or close+reopen
                long long open_ = (dp[i-1][j][0] + dp[i-1][j][1]) % MOD;
                if (j >= 1) open_ = (open_ + dp[i-1][j-1][1]) % MOD; // close + reopen
                dp[i][j][1] = open_;
            }
        }
        return (int)dp[n][k][0];
    }
};

int main() {
    int n = 4, k = 2;
    Solution sol;
    cout << sol.numberOfSets(n, k) << endl; // Expected output: 5
    return 0;
}
```

■ Python

```
MOD = 10**9 + 7

class Solution:
    def numberOfSets(self, n, k):
        # dp[i][j][open]: after processing the first i points, j segments CLOSED,
```

```

# open = 1 if currently inside an unclosed segment.
dp = [[[0, 0] for _ in range(k + 1)] for _ in range(n + 1)]
dp[0][0][0] = 1

for i in range(1, n + 1):
    for j in range(k + 1):
        # FREE at point i: point unused (stay free), or close, nothing new
        free_ = dp[i-1][j][0]
        if j >= 1:
            free_ = (free_ + dp[i-1][j-1][1]) % MOD
        dp[i][j][0] = free_

        # IN SEGMENT at point i: start new, pass through, or close+reopen
        open_ = (dp[i-1][j][0] + dp[i-1][j][1]) % MOD
        if j >= 1:
            open_ = (open_ + dp[i-1][j-1][1]) % MOD
        dp[i][j][1] = open_

return dp[n][k][0]

if __name__ == "__main__":
    n, k = 4, 2
    print(Solution().numberOfSets(n, k))    # Expected output: 5

```

■ JavaScript

```

const MOD = 1000000007;

class Solution {
    numberOfSets(n, k) {
        // dp[i][j][open]: after processing the first i points, j segments CLOSED,
        // open = 1 if currently inside an unclosed segment.
        const dp = Array.from({ length: n + 1 }, () =>
            Array.from({ length: k + 1 }, () => [0, 0])
        );
        dp[0][0][0] = 1;

        for (let i = 1; i <= n; i++) {
            for (let j = 0; j <= k; j++) {
                // FREE at point i: point unused (stay free), or close, nothing new
                let free_ = dp[i-1][j][0];
                if (j >= 1) free_ = (free_ + dp[i-1][j-1][1]) % MOD;
                dp[i][j][0] = free_;

                // IN SEGMENT at point i: start new, pass through, or close+reopen
                let open_ = (dp[i-1][j][0] + dp[i-1][j][1]) % MOD;
                if (j >= 1) open_ = (open_ + dp[i-1][j-1][1]) % MOD;
                dp[i][j][1] = open_;
            }
        }
        return dp[n][k][0];
    }
}

const n = 4, k = 2;
console.log(new Solution().numberOfSets(n, k)); // Expected output: 5

```

Complexity: $O(n \cdot k)$ states, each computed in $O(1)$ — $O(n \cdot k)$ time overall, comfortably fast for n, k up to 1000 (about 10^6 states). The table itself uses $O(n \cdot k)$ space, storing every point's values even though only the immediately preceding point is ever actually needed for the next step.

Approach 3 — Optimal (Roll the DP Down to $O(k)$ Space)

Idea: Keep the exact same recurrence as Approach 2, but notice that computing point i 's values only ever needs point $i-1$'s values — never anything further back. Replace the full 3-D table with just two length- $(k+1)$ arrays representing "the previous point's FREE and IN-SEGMENT values", overwritten fresh at each step.

Steps

- Keep two arrays, `free_[0..k]` and `open_[0..k]`, initialized so `free_[0] = 1` and everything else 0 (the state before any points are processed).
- For each of the n points: compute brand-new `nFree` and `nOpen` arrays purely from the current `free_` and `open_` (the previous point's values), using the same transition rules as Approach 2.
- Replace `free_`, `open_` with `nFree`, `nOpen` and move to the next point.
- After all n points, `free_[k]` is the answer.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;

class Solution {
public:
    int numberOfSets(int n, int k) {
        // Roll the DP down to just "previous point" and "current point" arrays.
        vector<long long> free_(k + 1, 0), open_(k + 1, 0);
        free_[0] = 1;

        for (int i = 1; i <= n; i++) {
            vector<long long> nFree(k + 1, 0), nOpen(k + 1, 0);
            for (int j = 0; j <= k; j++) {
                nFree[j] = free_[j];
                if (j >= 1) nFree[j] = (nFree[j] + open_[j-1]) % MOD;

                nOpen[j] = (free_[j] + open_[j]) % MOD;
                if (j >= 1) nOpen[j] = (nOpen[j] + open_[j-1]) % MOD;
            }
            free_ = nFree;
            open_ = nOpen;
        }
        return (int)free_[k];
    }
};

int main() {
    int n = 4, k = 2;
    Solution sol;
    cout << sol.numberOfSets(n, k) << endl;    // Expected output: 5
    return 0;
}
```

■ Python

```
MOD = 10**9 + 7

class Solution:
    def numberOfSets(self, n, k):
        # Roll the DP down to just "previous point" and "current point" arrays.
        free_ = [0] * (k + 1)
        open_ = [0] * (k + 1)
        free_[0] = 1
```

```

    for _ in range(n):
        nFree = [0] * (k + 1)
        nOpen = [0] * (k + 1)
        for j in range(k + 1):
            nFree[j] = free_[j]
            if j >= 1:
                nFree[j] = (nFree[j] + open_[j-1]) % MOD

            nOpen[j] = (free_[j] + open_[j]) % MOD
            if j >= 1:
                nOpen[j] = (nOpen[j] + open_[j-1]) % MOD
        free_, open_ = nFree, nOpen

    return free_[k]

if __name__ == "__main__":
    n, k = 4, 2
    print(Solution().numberOfSets(n, k))    # Expected output: 5

```

■ JavaScript

```

const MOD = 1000000007;

class Solution {
    numberOfSets(n, k) {
        // Roll the DP down to just "previous point" and "current point" arrays.
        let free_ = new Array(k + 1).fill(0);
        let open_ = new Array(k + 1).fill(0);
        free_[0] = 1;

        for (let i = 0; i < n; i++) {
            const nFree = new Array(k + 1).fill(0);
            const nOpen = new Array(k + 1).fill(0);
            for (let j = 0; j <= k; j++) {
                nFree[j] = free_[j];
                if (j >= 1) nFree[j] = (nFree[j] + open_[j-1]) % MOD;

                nOpen[j] = (free_[j] + open_[j]) % MOD;
                if (j >= 1) nOpen[j] = (nOpen[j] + open_[j-1]) % MOD;
            }
            free_ = nFree;
            open_ = nOpen;
        }
        return free_[k];
    }
}

const n = 4, k = 2;
console.log(new Solution().numberOfSets(n, k));    // Expected output: 5

```

Complexity: Still $O(n \cdot k)$ time (identical transitions, same total work), but only $O(k)$ extra space instead of $O(n \cdot k)$ — a significant memory reduction for the same n, k up to 1000, and the standard "roll the table" optimization whenever a DP's recurrence only ever looks one step back.

Side-by-Side Comparison

1. Brute Force	Enumerate every combination of k segments, check pairwise non-overlap	Combinatorially explosive — $O(2^k)$ per position	$O(n \cdot 2^k)$
2. Better	Full DP table over (point, segments closed, free/open)	$O(n \cdot k)$	$O(n \cdot k)$
3. Optimal	Same DP, rolled down to two $O(k)$ arrays	$O(n \cdot k)$	$O(k)$

Key Takeaways to Remember

- **"Count the number of ways to select k non-overlapping intervals"** is a strong signal for a left-to-right DP with a small extra state tracking whether you're currently inside an interval — not for direct combination enumeration.
- When intervals are **allowed to share endpoints**, model it explicitly as a transition ("close here AND reopen here") rather than trying to bolt it on as a special case afterward — it falls out naturally from the state machine.
- A well-chosen state machine can make invalid configurations (like overlapping segments) **structurally impossible to represent**, removing the need for any separate validity check.
- Whenever a DP's transition for step i only depends on step i-1, you can almost always **roll the table down** to two reusable arrays instead of storing every step — a simple, general memory optimization.
- All three tiers were validated against **Example 1** ($n = 4, k = 2 \rightarrow 5$) across C++, Python, and JavaScript, and checked against Examples 2 and 3 ($n=3,k=1 \rightarrow 3$ and $n=30,k=7 \rightarrow 796297179$) as well.